TEAM REVIEW EDITION

WS-IMP-001: Workstream v0.1 Coding-Agent Implementation Specification

A locked implementation-handoff specification prepared for structured team review.

DOCUMENT	WS-IMP-001
STATUS	Locked for implementation handoff
MATURITY	Implementation contract derived from approved design candidates; runtime proof remains required
VERSION / DATE	0.1 / 2026-07-11
OWNER	Workstream Engineering
SCOPE	Python/FastAPI implementation of the Workstream v0.1 architecture baseline, review and revision lifecycle, contribution records, compensation boundary, identity integration, evidence ports, asynchronous jobs, tests and proof artifacts

Review instruction: validate domain decisions, invariants, transaction boundaries, failure behaviour, and conformance coverage. Proposed changes should identify the exact section and affected invariant.

Contents

Use the PDF outline or the section index below to navigate the specification.

1. Purpose

This document is the deterministic execution contract for the coding agent implementing Workstream v0.1.

It converts the architecture and lifecycle documents into:

- a fixed technology and module direction;
- a source-precedence order;
- mandatory repository-integration rules;
- migration and persistence requirements;
- application-service and transaction boundaries;
- API, worker, identity and adapter requirements;
- an ordered implementation plan;
- conformance, concurrency, recovery and live-drill gates;
- explicit prohibited deviations;
- a definition of done based on evidence, not code presence.

This document does not authorize the coding agent to redesign the product. Where a normative parent contains more detail, that parent remains controlling within its scope.

---

2. Normative Precedence

The coding agent MUST resolve requirements in this order:

1. `WS-CON-001` controls the contribution, compensation-policy, award, outbox-fulfillment and callback boundary.
2. `WS-REV-001` controls reviewer authority, queue routing, leases, immutable review/revision chains and reject behavior.
3. `WS-ARCH-001` controls system boundaries, C4 components, technology, external ports, security, deployment and operational architecture.
4. Existing runtime-proven intake behavior controls implementation compatibility through `review_pending`.
5. Existing repository conventions control names and file placement only when they do not conflict with items 1-4.

If two requirements appear inconsistent, the coding agent MUST:

1. preserve the current proven intake path;
2. apply the narrower normative specification to its own scope;
3. document the apparent conflict in the implementation report;
4. stop before inventing a new business rule.

The coding agent MUST NOT choose a convenient behavior merely because the source code already contains a placeholder or incomplete implementation.

---

3. Locked Technology Decisions

The following choices are final for Workstream v0.1.

Concern	Required implementation
Language	Python only for lifecycle and application code
Web API	FastAPI with Pydantic request/response contracts
Persistence	PostgreSQL with SQLAlchemy and repository/unit-of-work boundaries matching the existing codebase
Schema migrations	The migration framework already present in the repository; Alembic when the current stack uses Alembic
Background work	Celery workers
Broker	Redis, used only for transient delivery/coordination
Checker execution	Existing isolated checker runner/process boundary
Canonical state	PostgreSQL
External delivery	Transactional outbox with at-least-once delivery and idempotent business effects
Production evidence	`FlowNodeAdapter` behind `ArtifactStorePort` and `SemanticIndexPort`
Development/CI evidence	`LocalStorageAdapter` or in-memory test adapter behind the same ports
Identity	Identity Issuer token/JWKS boundary plus local Workstream actor and role resolution
Testing	Existing Python test runner, with PostgreSQL integration tests for locking/constraint behavior

The coding agent MUST NOT introduce Rust, PyO3, a second API framework, another canonical database, Kafka, a new workflow engine, a provider-specific payment SDK in the domain, or new microservices for this implementation.

4. Mandatory Repository Reconnaissance

Before changing code, the coding agent MUST produce a short repository map in its implementation notes.

It MUST identify:

1. the FastAPI application entry point;
2. versioned router registration;
3. Pydantic request and response packages;
4. SQLAlchemy models and repository pattern;
5. unit-of-work or transaction helpers;
6. migration directory and current migration head;
7. Celery application, queues and task registration;
8. checker runner integration;
9. authentication/token dependencies;
10. authorization helpers;
11. structured error envelope;
12. audit-event mechanism;
13. outbox or event infrastructure already present;
14. existing project, task, assignment, submission, checker and actor models;
15. test fixtures for PostgreSQL, Redis and authenticated actors.

Repository reconnaissance is read-only. The coding agent MUST NOT begin by creating a parallel `app`, `core`, `domain`, `api_v2` or standalone service tree.

If the repository is not available, the coding agent MUST stop and report `source_repository_missing`. It MUST NOT generate a speculative replacement application.

5. Integration Rules

5.1 Preserve the proven intake spine

The existing path through `review_pending` is an integration boundary, not a rewrite invitation.

The coding agent MUST preserve:

- ProjectGuideSnapshot creation;
- sufficiency and policy-derivation jobs;
- manager approval of the exact effective policy;
- task locked-policy context;
- failed pre-submit behavior with no submission side effect;
- submission finalization;
- durable CheckerRun creation and result persistence;
- successful transition to `review_pending`.

The new implementation begins by attaching the locked review lifecycle to the successful checker-admission transaction or its existing durable continuation point.

5.2 No duplicate domain models

When an existing object represents the same business identity, the coding agent MUST extend or migrate it rather than create a second competing model.

Examples:

- extend the existing assignment relationship to support frozen compensation reference and task-scoped ban;
- map the existing finalized submission to `SubmissionVersion` if the identity and immutability contract can be preserved;
- reuse the existing audit/outbox framework when it meets the normative contract;
- do not create `NewTask`, `ReviewTaskV2`, `PaymentRecord` or a second contributor table.

5.3 Thin transport layer

FastAPI routers MUST:

- validate input;
- resolve identity and authority through dependencies;
- invoke one application command or query;
- translate the result into the stable response/error contract.

Routers MUST NOT contain state-machine transitions, SQL queries, queue-ordering logic, compensation evaluation or direct external calls.

5.4 Domain isolation

Review, contribution and compensation domain modules MUST NOT import:

- FastAPI request objects;
- Celery task objects;
- Redis clients;
- Flow Node HTTP clients;
- compensation-provider clients;
- UI types.

External behavior enters through ports implemented in infrastructure modules.

6. Logical Module Map

The coding agent MUST map the following logical components into the existing package structure. Names may follow repository conventions, but ownership MUST remain recognizable.

```
workstream
├── api
│   ├── dependencies          # token, actor, role and request context
│   ├── review_routes
│   ├── compensation_policy_routes
│   ├── contribution_routes
│   ├── compensation_award_routes
│   └── integration_callback_routes
│
├── application
│   ├── review_commands
│   ├── review_queries
│   ├── contribution_commands
│   ├── compensation_policy_commands
│   ├── compensation_queries
│   ├── artifact_commands
│   └── reconciliation_commands
│
├── domain
│   ├── authority
│   ├── review
│   ├── contribution
│   ├── compensation
│   ├── artifacts
│   └── events
│
├── persistence
│   ├── models
│   ├── repositories
│   └── unit_of_work
│
├── ports
│   ├── artifact_store
│   ├── semantic_index
│   └── compensation_fulfillment
│
├── adapters
│   ├── flow_node
│   ├── local_storage
│   └── compensation_http
│
├── workers
│   ├── review_sweeps
│   ├── checker_jobs
│   ├── outbox_dispatch
│   ├── reconciliation
│   └── projection_rebuild
│
└── tests
    ├── unit
    ├── integration
    ├── concurrency
    ├── contract
    └── live_drill
```

This is a responsibility map, not authorization to move stable modules unnecessarily.

7. Delivery Phases

The coding agent MUST implement and verify phases in the order below. It MUST keep the repository testable after every phase.

Phase	Required result	Exit gate
0	Reconnaissance, mapping and migration plan	Repository map and no unresolved identity duplication
1	Shared identity, actor, authority and transaction foundations	Identity/authorization tests pass without breaking intake
2	Review/revision schema and persistence	Migrations, constraints and repository tests pass on PostgreSQL

Phase	Required result	Exit gate
3	Review queue, claims, leases, decisions and recovery	WS-REV-001 conformance, concurrency and live drill pass
4	Contribution and compensation-policy schema/persistence	WS-CON-001 persistence and policy tests pass
5	Atomic review-to-contribution/award transaction	Creation-matrix, replay and race tests pass
6	Outbox delivery and fulfillment callback	Adapter contract, callback and reconciliation tests pass
7	Flow Node ports and production adapter	Evidence ingest/verify/pin/search/recovery tests pass
8	Read APIs, operations UI support and observability	Authorized read-model and operational metrics tests pass
9	Full internal-loop drill and release evidence	End-to-end proof pack accepted

No phase is complete because code compiles. The stated exit gate is mandatory.

8. Phase 1 - Identity, Actor and Authority Foundation

8.1 Token verification

Implement one FastAPI authentication dependency that:

1. extracts the bearer token;
2. validates the configured issuer;
3. validates the Workstream audience;
4. validates signature through cached issuer JWKS;
5. validates `exp`, `nbf` and other required temporal claims using configured clock skew;
6. obtains the issuer subject;
7. resolves `(issuer, subject)` to one active local Workstream actor;
8. attaches actor and correlation context to the request.

Failure codes MUST distinguish invalid token, unknown actor and inactive actor without revealing other users.

The implementation MUST support safe JWKS key overlap and refresh. A stale cached key may be used only according to configured issuer policy; the coding agent MUST NOT silently bypass signature verification during issuer unavailability.

8.2 Local role resolution

The local role store MUST load:

- active admin roles;
- active project scopes for those roles where applicable;
- active `ProjectRoleGrant` records;
- task assignment status for task-scoped commands.

The token MUST NOT be treated as authoritative for `submitter`, `reviewer`, `project_manager`, `operator`, `finance_authority` or `reputation_authority`.

8.3 Authorization service

Implement command-specific guards for at least:

- project administration;
- role-grant creation and revocation;
- submitter task claim/finalization;
- reviewer queue read;
- review claim/release/decline/decision;
- compensation-policy administration;
- adapter-binding administration;
- contribution and award reads;
- fulfillment callback service actor.

Every guard MUST receive the local actor and exact resource/project identifiers. Generic `is_admin` checks are insufficient for resource-scoped commands.

8.4 Service principals

Flow Node and compensation-adapter integration MUST use configured service actors/credentials distinct from human request identity.

A human bearer token MUST never be forwarded to an external system.

---

9. Phase 2 - Review and Revision Persistence

Implement the complete `WS-REV-001` object model, including:

- `ProjectRoleGrant` and qualification snapshot reference;
- `SubmissionVersion` and prior-version chain;
- `SubmissionFindingResponse`;
- `ReviewQueueEntry`;
- `ReviewLease` including permanent completed/expired/released attempts;
- `Review` and prior-review chain;
- `ReviewFinding`;
- `FindingResolution`;
- `TaskAssignment` task-scoped ban extension;
- project review-policy fields.

9.1 Required review constraints

The migration MUST enforce:

```
UNIQUE ReviewQueueEntry(submission_version_id)
UNIQUE Review(submission_version_id)
UNIQUE SubmissionVersion(task_id, version_number)
UNIQUE FindingResolution(finding_id, submission_version_id)

PARTIAL UNIQUE ReviewLease(review_queue_entry_id) WHERE status = 'active'
PARTIAL UNIQUE ReviewLease(reviewer_id) WHERE status = 'active'
```

The migration MUST also add the exact indexes and compatible-state check constraints required by `WS-REV-001` section 17.

9.2 Migration rules

Migrations MUST be forward-only and safe for the existing data set.

For every new non-null field on existing rows, the coding agent MUST provide an explicit deterministic backfill derived from authoritative records. It MUST NOT use a random policy version, current time as historical business time, a placeholder actor, or a fabricated review decision.

If existing data cannot satisfy a required invariant, the migration MUST stop with a precise report and a separate audited data-remediation plan.

9.3 Enum and state compatibility

Database enums or check constraints MUST represent the exact v0.1 values from `WS-REV-001`. The coding agent MUST NOT add `in_progress`, `approved`, `escalated`, `adjudication_pending` or other undocumented review decisions/states.

10. Phase 3 - Review Queue, Claim and Decision Services

10.1 Checker admission to queue creation

When a durable checker admits a finalized SubmissionVersion:

1. transition the task to `review_pending` according to the existing intake state machine;
2. create exactly one ReviewQueueEntry for the SubmissionVersion;
3. create it in the routing mode and timestamps required by `WS-REV-001`;
4. append the queue-created audit event;
5. commit all effects atomically or through the already-proven durable boundary without allowing an orphaned admitted version.

An exact retry MUST return or observe the existing queue entry.

10.2 Reviewer queue query

Implement the reviewer view as three distinct sections:

```
{
  "active_lease": null,
  "preferred_backlog": [],
  "open_queue": []
}
```

Ordering MUST follow `WS-REV-001`:

- preferred backlog: the normative preferred order;
- open queue: FIFO using the normative queue-age field and stable ID tie-breaker;
- expired preference becomes open without resetting first queue age;
- expired or released lease returns to open queue with the normative requeue timestamp behavior.

Queue totals MUST not reveal unauthorized project work.

10.3 Claim transaction

The claim application service MUST execute one PostgreSQL transaction:

1. obtain database time;
2. lock the selected ReviewQueueEntry;
3. perform lazy preference/lease recovery required for that entry;
4. verify the entry remains pending and visible to the claimant;
5. verify active reviewer/both ProjectRoleGrant for the entry project;
6. verify the reviewer is not the submission creator;
7. verify the reviewer has no active global lease in v0.1;
8. verify preferred-routing rules;
9. freeze the reviewer compensation-policy version required by `WS-CON-001`;
10. insert one active ReviewLease;
11. update queue routing state;
12. append audit event;
13. commit and return the lease.

Database partial uniqueness is the final race guard. A uniqueness conflict MUST map to the stable conflict code, not a 500 error.

10.4 Release and decline

Manual release MUST:

- authorize only the active leased reviewer or a separately specified operator override;
- close the lease as released;
- return the entry to open routing;
- clear preferred stickiness according to `WS-REV-001`;
- record `ReviewerReleasedTask` rather than `ReviewerLeaseExpired`;
- commit in one transaction.

Declining a preferred entry before claim MUST implement the exact preferred-routing transition and authority rule from `WS-REV-001`.

10.5 Decision transaction

The decision endpoint MUST use an idempotency key and one database transaction.

The service MUST:

1. lock the ReviewLease and ReviewQueueEntry;
2. use database time to verify the lease is still active and unexpired;
3. recheck the reviewer's active project grant;
4. lock the SubmissionVersion, task and TaskAssignment required by the decision;
5. validate the decision payload and finding rules;
6. insert exactly one immutable Review;
7. insert immutable findings and prior-finding resolutions;
8. close the queue entry and consume the lease;

9. apply accept, needs_revision or reject task effects;
10. execute the atomic contribution/award work after Phase 5 is installed;
11. append audit and outbox records;
12. commit and return the complete committed result.

No decision path may update a prior SubmissionVersion, Review or ReviewFinding.

10.6 Timer workers

Implement named Celery jobs for:

- preference expiry;
- lease expiry;
- orphan queue/lease reconciliation.

Workers MUST claim batches with PostgreSQL locking suitable for multiple worker replicas, normally `FOR UPDATE SKIP LOCKED` over deterministic indexed candidates.

Sweep and request-time lazy recovery MUST invoke the same application transition service.

Redis time MUST NOT decide expiry. Use PostgreSQL/database time.

---

11. Review API Requirements

Implement the normative capability set:

```
POST /v1/projects/{project_id}/role-grants
GET  /v1/projects/{project_id}/role-grants
POST /v1/projects/{project_id}/role-grants/{grant_id}/revoke

GET  /v1/reviews/queue/mine
GET  /v1/projects/{project_id}/reviews/queue

POST /v1/reviews/queue/{queue_entry_id}/claim
POST /v1/reviews/leases/{lease_id}/release
POST /v1/reviews/queue/{queue_entry_id}/decline-preference
POST /v1/reviews/leases/{lease_id}/decision

GET  /v1/tasks/{task_id}/review-chain
GET  /v1/submission-versions/{version_id}/review
```

Endpoint names may be aligned with an existing consistent router convention only if capability boundaries, payloads, responses and error semantics remain identical and the change is documented.

No delete or edit endpoint may exist for SubmissionVersion, Review, ReviewFinding or FindingResolution.

---

12. Phase 4 - Contribution and Compensation Persistence

Implement the complete `WS-CON-001` object model:

- `ContributionRecord`;
- `CompensationPolicy`;
- `CompensationPolicyVersion`;
- `CompensationRule`;
- `CompensationAwardDefinition`;

- frozen compensation references on TaskAssignment and ReviewLease;
- `CompensationAward`;
- `ProjectCompensationAdapterBinding`;
- `CompensationFulfillmentReceipt`;
- `CompensationStatusProjection`;
- required outbox event data.

12.1 Required compensation constraints

The database MUST enforce all `WS-CON-001` section 20 constraints, including:

```
UNIQUE ContributionRecord(source_review_id, contribution_type)
UNIQUE CompensationPolicyVersion(compensation_policy_id, version_number)
PARTIAL UNIQUE CompensationPolicy(project_id) WHERE status = 'active'
UNIQUE CompensationRule(compensation_policy_version_id, contribution_type)
UNIQUE CompensationAwardDefinition(compensation_policy_version_id, contribution_type,
instrument_type)
UNIQUE CompensationAward(contribution_record_id, instrument_type)
PARTIAL UNIQUE ProjectCompensationAdapterBinding(project_id, instrument_type) WHERE status
= 'active'
UNIQUE CompensationFulfillmentReceipt(adapter_binding_id, external_event_id)
PARTIAL UNIQUE CompensationFulfillmentReceipt(compensation_award_id) WHERE reported_status
= 'fulfilled'
PRIMARY KEY CompensationStatusProjection(compensation_award_id)
```

Quantity checks, enum checks, project-chain foreign keys, frozen-policy non-null fields and all normative indexes are mandatory.

All monetary/point quantities MUST use exact decimal storage. Python `float` MUST NOT be used for policy or award quantities.

12.2 Compensation policy service

Implement immutable version behavior:

- creating a policy creates an empty version 1 draft;
- creating the next version copies the current published version into a new draft;
- PUT replaces the complete draft rule set;
- publish validates the entire draft and requires `expected_current_version_id`;
- published and retired versions are immutable;
- one project has at most one active policy;
- a project may intentionally publish an explicit unpaid rule;
- assignment and review lease freeze one committed active policy version.

Publishing a new policy MUST NOT change existing TaskAssignment, ReviewLease, ContributionRecord or CompensationAward terms.

13. Phase 5 - Atomic Review-to-Contribution and Award Operation

The contribution operation is part of the same transaction that records the Review.

13.1 Creation matrix

Review decision	Reviewer ContributionRecord	Submitter ContributionRecord
`accept`	Create	Create
`needs_revision`	Create	Do not create
`reject`	Create	Do not create

The reviewer contribution is `completed_review` and is created for every valid recorded decision regardless of outcome.

The submitter contribution is `accepted_submission` and is created only for `accept`.

13.2 Deterministic award evaluation

For each new ContributionRecord:

1. load the frozen CompensationPolicyVersion reference from the originating TaskAssignment or ReviewLease;
2. select the exact rule for the contribution type;
3. if the rule is unpaid, create no CompensationAward and record the policy reference on the contribution as required;
4. if compensated, create one CompensationAward for every award definition;
5. copy immutable instrument, unit, quantity, policy version and adapter binding references into the award;
6. append one `CompensationAwardCreated` event per award;
7. append one `CompensationFulfillmentRequested` event per fulfillable award;
8. append `ContributionRecorded` for each contribution;
9. commit all records with the Review.

Decision outcome MUST NOT change the reviewer's award unless the published policy itself contains a normative contribution-type distinction. v0.1 uses `completed_review`, not separate accepted/rejected review contribution types.

13.3 Replay behavior

An exact replay of a committed decision MUST return:

- the existing Review;
- the existing reviewer ContributionRecord;
- the existing submitter ContributionRecord when accepted;
- the existing CompensationAwards;
- no new outbox events.

The decision idempotency record MUST protect the entire combined result. The same key with a different payload returns `409 idempotency_mismatch`.

14. Compensation and Contribution APIs

Implement the normative API groups from `WS-CON-001` section 18.

14.1 Compensation policies

```
POST /v1/projects/{project_id}/compensation-policies
POST /v1/projects/{project_id}/compensation-policies/{policy_id}/versions
PUT  /v1/projects/{project_id}/compensation-policy-versions/{version_id}/draft
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/publish
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/retire
GET  /v1/projects/{project_id}/compensation-policies
GET  /v1/projects/{project_id}/compensation-policy-versions/{version_id}
```

14.2 Adapter bindings

```
POST /v1/projects/{project_id}/compensation-adapter-bindings
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/suspend
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/resume
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/retire
GET  /v1/projects/{project_id}/compensation-adapter-bindings
```

14.3 Contribution reads

```
GET /v1/contributions/{contribution_record_id}
GET /v1/contributors/me/contributions
GET /v1/contributors/{contributor_id}/contributions
GET /v1/projects/{project_id}/contributions
GET /v1/projects/{project_id}/tasks/{task_id}/contributions
```

Ordering MUST be:

```
recorded_at DESC, id DESC
```

No create, update or delete ContributionRecord endpoint may exist.

14.4 Award reads and fulfillment callback

```
GET  /v1/compensation-awards/{award_id}
GET  /v1/projects/{project_id}/compensation-awards
GET  /v1/contributors/{contributor_id}/compensation-awards
POST /v1/integrations/compensation/fulfillment-reports
```

The callback is the only external mutation endpoint in this boundary. It may create a receipt and update the replaceable status projection. It may not change a contribution, award, policy, assignment, lease, Review, task or contributor.

15. Phase 6 - Transactional Outbox and Fulfillment Adapter

15.1 Outbox persistence

Every outbox record MUST contain:

- immutable `event_id`;
- `event_type`;
- independent `event_version`;
- aggregate type and ID;
- project ID where applicable;

- occurred-at timestamp;
- correlation ID;
- stable idempotency key;
- schema-valid non-sensitive payload;
- delivery state, attempt count and next-attempt time.

`event_version: 1` is an event-schema version and does not mean Workstream v1.0.

15.2 Dispatcher

The Celery dispatcher MUST:

1. select due outbox rows in deterministic batches using PostgreSQL locking suitable for multiple replicas;
2. resolve the active adapter binding referenced by the immutable award;
3. deliver the normative payload with event ID and idempotency key unchanged across retries;
4. record each attempt outcome;
5. mark durable acknowledgement when received;
6. back off transient failure;
7. expose exhausted/dead-letter work to operations;
8. never create another award or replacement event because delivery failed.

15.3 Callback authentication

The callback MUST authenticate the adapter service actor and verify:

- active binding;
- project match;
- instrument match;
- award identity;
- external event identity;
- request authenticity/replay controls;
- allowed terminal transition.

Credentials and secrets MUST come from runtime secret configuration. `route_key` is routing metadata and MUST NOT contain a secret.

15.4 Callback state rules

Implement exactly:

- new valid receipt: `201`;
- exact idempotent replay: `200` and existing receipt;
- malformed payload: `400`;
- invalid service identity: `401`;
- binding/project/instrument mismatch: `403`;
- unknown award: `404`;
- idempotency or contradictory-terminal conflict: `409`;
- invalid fulfillment semantics: `422`.

When failed and fulfilled callbacks race, fulfilled is the monotonic final state. A late acknowledgement MUST never regress fulfilled state.

16. Phase 7 - Flow Node Evidence Ports

16.1 Ports

Define domain-facing protocols for:

```
class ArtifactStorePort(Protocol):
    def request_ingest(...): ...
    def get_receipt(...): ...
    def verify(...): ...
    def pin(...): ...
    def retrieve(...): ...

class SemanticIndexPort(Protocol):
    def index(...): ...
    def search(...): ...
```

The actual signatures MUST use repository DTO conventions and async/sync style consistently. The ports MUST not expose a provider HTTP response object to the domain.

16.2 ArtifactBinding

Persist a Workstream `ArtifactBinding` containing at least:

- Workstream source reference;
- artifact/content ID;
- manifest ID;
- expected and verified digest;
- verification state;
- pin state;
- provider/provenance reference;
- last reconciliation timestamp;
- immutable receipt references where required.

Required bindings MUST be verified and pinned before the dependent lock/check/review gate.

16.3 FlowNodeAdapter

The production adapter MUST:

- use service authentication;
- use stable idempotency keys;
- validate returned identifiers and digest evidence;
- translate network/provider errors into typed infrastructure errors;
- persist/reconcile receipts through the application service;
- never update task/review state directly.

16.4 LocalStorageAdapter

The development/CI adapter MUST implement the same port semantics needed by tests.

Application startup MUST fail if `LocalStorageAdapter` is selected in the production environment.

16.5 Search authorization

Flow Node search results are candidates. Before returning any result, Workstream MUST re-authorize the requesting actor for the referenced project/resource and remove unauthorized candidates without revealing their count or metadata.

Flow Node unavailability MUST become evidence-unavailable/retry state. It MUST NOT become checker failure attributed to the contributor or a human reject decision.

---

17. Transaction and Locking Rules

17.1 Unit of work

Every mutation application service MUST use one explicit unit of work.

The unit of work MUST include:

- locked reads required for the invariant;
- canonical inserts/updates;
- audit facts;
- caused outbox events;
- idempotency record where applicable.

External network calls MUST occur after commit through workers/outbox unless a normative read-only verification call is explicitly required. No external call may be allowed to hold a review-decision or policy-activation database transaction open.

17.2 Database time

Lease and preference expiry MUST use PostgreSQL time inside the deciding transaction. Web-server and worker clocks are observability inputs only.

17.3 Row locking

Use targeted `SELECT ... FOR UPDATE` or the SQLAlchemy equivalent for:

- ReviewQueueEntry claim;
- ReviewLease decision/release/expiry;
- task and assignment terminal decision effects;
- active compensation-policy selector during publish/freeze;
- CompensationAward/status during callback;
- idempotency record creation/resolution.

Use `SKIP LOCKED` only for background batch claiming. Do not use it to make a user claim silently skip the requested queue entry.

17.4 Isolation and constraints

The default transaction isolation may remain the repository standard when row locks and constraints make the operation correct. The coding agent MUST not raise global isolation to serializable as a substitute for required row locks and uniqueness.

Every database `IntegrityError` caused by an expected race MUST be translated to the normative idempotent result or stable conflict. Unexpected integrity errors MUST be logged with correlation context and returned as the existing safe internal error.

18. Error Contract

All APIs MUST use the existing structured error envelope. If the repository lacks one, implement:

```
{
  "error": {
    "code": "reviewer_capacity_reached",
    "message": "The reviewer already has an active review lease.",
    "details": {},
    "correlation_id": "uuid",
    "retryable": false
  }
}
```

Machine codes from `WS-REV-001` and `WS-CON-001` are stable public contracts.

The coding agent MUST NOT return raw SQL, stack traces, token claims, provider responses or secret configuration.

Expected uniqueness races MUST not surface as `500`.

19. Audit Requirements

Audit facts MUST be appended for every authority or lifecycle action required by the normative specifications.

Each fact MUST include:

- event type;
- occurred-at database timestamp;
- actor ID and actor kind;
- effective local role/grant reference where applicable;
- project ID;
- exact resource IDs;
- correlation ID;
- idempotency key where applicable;
- before/after state names for mutable operational records;
- reason for admin override/revocation;
- immutable policy/version references in force.

Application logs are not a replacement for audit facts.

The coding agent MUST preserve distinct audit meanings for voluntary reviewer release and lease timeout.

20. Worker and Retry Configuration

Create named queues or routing keys using existing Celery conventions for:

- project setup/checkers;

- review timers;
- outbox delivery;
- evidence reconciliation;
- compensation reconciliation;
- projection rebuild.

Every task MUST:

- accept a stable record/job identifier rather than a full mutable object snapshot;
- reload current PostgreSQL state;
- be safe under duplicate delivery;
- record durable retry/attempt state where required;
- acknowledge after durable outcome;
- emit structured correlation context.

Retry policy MUST classify errors:

- validation/authority/domain conflicts are terminal and are not retried;
- network timeout, connection and provider 5xx are retryable;
- provider 4xx is terminal unless the adapter contract explicitly marks it retryable;
- unknown exception follows bounded retry then operational dead-letter handling.

The exact intervals are deployment configuration, but the algorithm and maximum-attempt handling MUST be tested.

---

21. Read Models and Pagination

Read services MAY compose canonical records with replaceable projections. They MUST label projection state and MUST not use it as sole mutation authority.

Cursor pagination MUST use stable sort keys.

Contribution history uses:

```
recorded_at DESC, id DESC
```

Queue views use the normative routing timestamp plus ID tie-breaker.

Authorization filtering occurs before totals and cursors are exposed. Unauthorized records MUST not be inferable through counts, empty-page behavior or error differences.

Projection rebuild commands MUST be idempotent and must derive state from canonical rows and receipts.

---

22. Security Requirements

The implementation MUST include:

- deny-by-default authorization;
- local role/grant revocation checks at command time;
- self-review prohibition;
- task-scoped banned-assignment check;

- separate human, worker and adapter principals;
- TLS for all production network boundaries;
- runtime secret storage;
- checker resource and credential isolation;
- sensitive-data exclusion from events and logs;
- parameterized SQL/ORM access;
- request body and artifact-reference validation;
- callback replay protection;
- rate limiting at ingress or the existing API-control layer for sensitive endpoints.

No provider secret, token or artifact byte payload may be stored in an outbox event.

---

23. Observability Requirements

All API and worker paths MUST propagate or create a correlation ID.

Structured logs and traces MUST include applicable identifiers:

- `project_id`;
- `task_id`;
- `submission_version_id`;
- `review_queue_entry_id`;
- `review_lease_id`;
- `review_id`;
- `contribution_record_id`;
- `compensation_award_id`;
- `outbox_event_id`.

Metrics MUST cover at least:

- API latency/error by route and error code;
- queue size and oldest age by preferred/open view;
- active lease count and expiry/release rates;
- claim conflict and reviewer-capacity conflicts;
- checker backlog, duration and infrastructure failure;
- outbox oldest age, attempts, success/failure and exhausted count;
- Flow Node ingest/verify/pin latency and unavailable bindings;
- reviews missing expected contribution records;
- awards by delivery/fulfillment state and pending age;
- contradictory callback attempts.

Alerts MUST be based on deployment-configured thresholds and have an operator runbook reference.

---

24. Test Strategy

The coding agent MUST add tests at four levels.

24.1 Unit tests

Unit-test pure rules:

- reviewer eligibility;
- routing-mode selection;
- preference and lease transitions;
- review payload/finding validation;
- decision effect matrix;
- compensation policy validation;
- award evaluation with exact decimals;
- callback monotonic state reduction;
- typed adapter error mapping.

Unit tests MUST not mock away database behavior that is itself the requirement.

24.2 PostgreSQL integration tests

Use real PostgreSQL for:

- partial unique indexes;
- composite/project-chain foreign keys;
- check constraints;
- row locks;
- database time;
- `SKIP LOCKED` workers;
- transaction rollback;
- migration upgrade from the prior head.

SQLite MUST NOT be used as the only test database for these features.

24.3 Concurrency tests

Run true concurrent transactions proving:

1. two reviewers claim one entry: exactly one succeeds;
2. one reviewer claims two entries: exactly one active lease exists;
3. decision races expiry: one normative terminal outcome exists;
4. duplicate decision: one Review and one contribution/award set exists;
5. two policy versions publish concurrently: one committed active selector is frozen by new work;
6. duplicate outbox dispatch: one adapter business effect under the stable key;
7. failed and fulfilled callback race: final state is fulfilled;
8. two fulfilled callbacks: one unique fulfilled receipt unless exact replay.

Tests MUST assert database row counts and immutable values, not only HTTP status.

24.4 Contract tests

Add JSON-schema or equivalent contract tests for:

- review API payloads and error codes;
- review-chain reads;
- contribution reads;
- compensation policy draft/publish;
- `ContributionRecorded`;
- `CompensationAwardCreated`;
- `CompensationFulfillmentRequested`;
- fulfillment callback;
- Flow Node adapter request/receipt mapping.

24.5 Security tests

Test:

- invalid issuer, audience, signature and expired token;
- unknown/inactive local actor;
- revoked role/grant;
- admin without contributor reviewer grant;
- reviewer self-review attempt;
- banned submitter task reclaim;
- unauthorized project queue/count leakage;
- human token forwarding prohibition;
- unauthorized adapter callback;
- callback replay and mismatch;
- LocalStorageAdapter rejected in production.

25. Required Live Drills

25.1 Review drill

The drill MUST use supported APIs and workers to prove:

1. first submission enters open FIFO;
2. authorized reviewer claims one lease;
3. second claim is blocked by capacity;
4. `needs_revision` creates immutable review/findings and reviewer contribution;
5. new SubmissionVersion explicitly responds to findings;
6. prior reviewer receives preferred backlog;
7. preference expiry opens the same entry without losing original FIFO age;
8. a different reviewer may claim after expiry;
9. accept creates reviewer and submitter contributions and awards;

10. replay creates nothing new;
11. reject in a separate task closes the task and bans that contributor-task assignment;
12. lease timeout returns work to open FIFO with distinct audit evidence.

25.2 Compensation drill

The drill MUST prove:

1. finance authority publishes a project policy with money and/or project points;
2. assignment and lease freeze the exact active policy version;
3. later policy publication does not alter frozen work;
4. review decision atomically creates contribution, awards and outbox events;
5. dispatcher redelivery retains one business effect;
6. adapter reports fulfilled;
7. exact callback replay returns the existing receipt;
8. contradictory callback is rejected;
9. contribution history is globally addressable and correctly project-filtered;
10. no internal payment request, wallet balance or point ledger record is created.

25.3 Evidence drill

The drill MUST prove:

1. submission artifact intent produces an outbox request;
2. Flow Node returns CID, manifest and verification/pin receipt;
3. required unverified or unpinned artifact blocks the dependent gate;
4. verified/pinned artifact admits checker/review;
5. digest mismatch is an evidence failure, not contributor reject;
6. provider unavailability pauses and later reconciles;
7. semantic search results are filtered by Workstream authorization.

25.4 Full internal-loop drill

Run one task from project guide through fulfilled adapter receipt without direct database edits.

Capture request/response IDs, canonical database snapshots, worker logs, audit events, outbox attempts and final immutable chain.

---

26. Implementation Evidence Pack

The coding agent MUST produce:

1. repository map and changed-file summary;
2. migration list and schema-diff summary;
3. generated OpenAPI diff;
4. test command list and pass/fail counts;
5. concurrency-test evidence;

6. review, compensation, evidence and full-loop drill reports;
7. example immutable review chain;
8. example contribution/award/receipt chain;
9. outbox replay evidence;
10. authorization-denial evidence;
11. remaining known limitations that are explicitly out of v0.1 scope;
12. rollback and deployment notes.

The evidence pack MUST state separately:

- what was implemented;
- what was tested;
- what was live-drilled;
- what remains designed only.

27. Prohibited Deviations

The coding agent MUST NOT:

1. introduce Rust or PyO3 for the v0.1 lifecycle;
2. create a second Workstream application or parallel database model;
3. add unrelated products or platform components;
4. move Workstream roles into Identity Issuer tokens;
5. make reputation automatically grant reviewer authority in v0.1;
6. allow admin role alone to review work;
7. allow self-review;
8. allow more than one active global reviewer lease in v0.1;
9. collapse preferred backlog and open FIFO into an unordered queue;
10. overwrite or delete submission, review, finding, contribution, award or receipt history;
11. add an adjudication layer;
12. reopen rejected tasks to another contributor;
13. pay the submitter for `needs_revision` or `reject` through a submitter ContributionRecord;
14. withhold reviewer contribution because the outcome is reject or needs_revision;
15. implement payment requests, provider attempts, wallets, balances or point ledgers inside Workstream;
16. call compensation or Flow Node providers inside the review-decision transaction;
17. treat Redis or a projection as canonical state;
18. use local filesystem as production artifact authority;
19. treat Flow Node failure as contributor failure or reject;
20. add undocumented enum values, states, endpoints or side effects;
21. weaken database constraints because service validation exists;
22. declare success without the required PostgreSQL races and live drills.

28. Definition of Done

Implementation is complete only when all statements below are true.

- Existing intake tests and the WS-POL-001-16 path remain green.
- Identity Issuer verification and local Workstream authorization are tested.
- All `WS-REV-001` schema, API, concurrency, timer, recovery, audit and conformance requirements pass.
- All `WS-CON-001` schema, policy, creation-matrix, award, outbox, callback, read and conformance requirements pass.
- FlowNodeAdapter satisfies evidence port, verification, pinning, search and recovery tests.
- Every canonical mutation commits with its audit facts and caused outbox events.
- Exact replays return existing business results and create no duplicate facts.
- PostgreSQL constraints prevent invalid duplicate or cross-project chains.
- Operations can observe queue age, lease expiry, checker backlog, outbox failure, evidence unavailability and award fulfillment.
- Full internal-loop drill completes through adapter receipt with no manual database edit.
- Documentation and OpenAPI match deployed behavior.
- The implementation evidence pack distinguishes runtime proof from design.

---

29. Coding-Agent Start Instruction

The implementation agent MUST begin with this sequence:

```
1. Read WS-ARCH-001, WS-REV-001 and WS-CON-001 completely.
2. Inspect the repository and produce the mandatory map from section 4.
3. Run the existing test suite and WS-POL-001-16 drill before changes.
4. Identify existing objects that will be extended; do not create duplicates.
5. Propose the ordered migration files and application-service mapping.
6. Implement Phase 1 only.
7. Run its exit gate and report evidence.
8. Continue phase by phase; do not skip a failed gate.
9. Stop and report a normative conflict instead of inventing a business rule.
10. Finish only after the full evidence pack and internal-loop drill exist.
```

The coding agent has authority to implement this specification inside the supplied Workstream repository. It does not have authority to change product scope, lifecycle decisions or external-system ownership.